A Certified Zero Removes a Computation Opcode

Smaller complete presentations of intuitionistic primitive recursive arithmetic

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

A scalar-iteration proof checker can use the initializer itself to distinguish computation from paired introduction. This eliminates a separate computation tag, provided direct computation is restricted to initializers with nonempty syntax. The restriction loses no theorem: the fixed term $C = I(2, 1; v.0)$ has a direct permitted proof of $C = 0$. Replacing a missing zero initializer, a missing zero predecessor, or both by C, and transporting this equality once, recovers every excluded computation equation. Each excluded computation leaf is replaced by three nodes. The numerical language, native formulas, and every native sequent are preserved exactly.

The inherited scalar-iteration and paired-binder presentation is stated explicitly, together with its arithmetic and logical reconstruction and both interpretation composites with full first-order iPRA. The new restriction therefore preserves whole-domain bi-interpretability and equiconsistency. Complete implementations, proof producers, positive and adversarial certificates, source inventories, and offline reproduction scripts accompany the report. Gzip level-nine sizes are reported only as auxiliary measurements.

The comparison covers all 13 current draft ZIPs at commit `fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`, including 370 distinct Python sources. Both the supplied frontier and stronger matched-driver controls are measured. One runtime-qualified implementation is below all three matched coordinate minima; the recommended explicit-list implementation has smaller source bytes and avoids the malformed-string qualification. The improvement is a proved restriction and a smaller checker, not an absolute minimality theorem or the requested major new arithmetic foundation.

What is established. A new admissible computation restriction, an eliminated packet opcode, complete smaller implementations, and identity preservation of the existing full-iPRA interpretation. **What remains open.** An absolute simplest axiomatization and a significant foundational breakthrough. The mathematical proofs are written proofs, not Lean or Rocq formalizations.

Contents

1 Result, comparison, and attribution 3

2 The complete native theory 4

3 Using the initializer to select computation 6

4 Full iPRA and both interpretation composites 8

5 The complete checker and its interpretation 10

6 Executable evidence and reproducibility 14

7 Two reductions that were rejected 14

8 A finite-model obstruction to eager initialization 15

9 Assessment and remaining limits 18

A Attribution for the complete inherited bridge 19

B Complementary introductions with one local assumption 19

C Arithmetic strength and both ordinary interpretation composites 24

D Internal scalar predecessor bootstrap 26

E From literal-equation induction to full intuitionistic PRA 30

F Pairing, primitive recursion, and internal numerical composites 35

1 Result, comparison, and attribution

The main change is small enough to explain without the encoding. Computation packets already need an initializer. Ensure that its syntax is nonempty, and its presence can select computation; paired-introduction packets leave that field empty. The difficulty is mathematical: zero is an indispensable initializer. The closed identity

$$\boxed{I(S(S0), S0; v.0) = 0}$$

solves that difficulty using one computation whose predecessor and initializer are both nonempty. No earlier zero-initializer theorem is used. Equality transport then restores the missing cases.

1.1 Exact whole-program comparison

The source snapshot is [1]. The recursive inventory contains 13 direct archives, 2,701 Python occurrences, and 370 distinct raw source hashes. All 31 distinct programs at or below at least one supplied coordinate minimum were read and classified: three are complete checkers, and 28 are supporting programs. Tiny build scripts and test drivers that call a checker are not counted as proof checkers.

The supplied coordinate minima are 268 significant normalized tokens, 300 AST nodes, and 616 normalized UTF-8 bytes. Different files attain them. The often quoted larger headline values in individual reports do not include every ancillary or matched implementation in this snapshot. The seven additional matched controls remove only the obsolete `print('REJECT'); raise` wrapper from selected affine and arity programs. These controls establish the stronger comparison envelope 264 tokens, 300 nodes, and 601 bytes. Their exact AST transformation, original bytes, hashes, and verification receipts are included.

Complete implementation	Tokens	AST	Bytes	Gzip-9
Supplied coordinate minima	268	300	616	—
Matched coordinate minima	264	300	601	—
New explicit list test	264	308	566	351
New byte profile	264	310	558	345
New joint profile (qualified)	262	296	598	350

All new raw-source lengths equal their normalized lengths. The two envelope rows are coordinatewise minima from different files; each new implementation row is one complete file.

`code/checker.py` is the recommended implementation: it uses an explicit `isinstance(a,list)` test. The byte profile uses the inherited exact-JSON-type predicate `a*0 == []`. The joint profile uses the inherited exception-based traversal; it has the smallest combined structural counts but a qualification on malformed strings. These are distinct complete programs. No row combines measurements from different programs.

The joint profile is strictly below every coordinate of both envelopes. That statement includes its runtime qualification; it must not be silently attributed to the recommended list-based checker. The recommended and byte profiles primarily improve source bytes. The observed envelopes are achieved upper bounds on an unknown optimum, not mathematical lower bounds on all theories or implementations.

1.2 Measurement convention

The whole module is counted: imports, JSON input, syntax and scope checks, arithmetic and logical rules, and the acceptance driver. Normalization removes genuine docstrings and comments through AST reconstruction, uses `ast.unparse`, and appends one newline. Tokens exclude whitespace, comments, indentation markers and end markers. AST counts include the full module. CPython 3.12.14 and zlib 1.3.2 were used; another implementation can change normalization or compression.

Both raw and normalized source are compressed by

```
gzip.compress(source_bytes, compresslevel=9, mtime=0)
```

including the gzip wrapper and no filename. The exact `.py.gz` streams accompany the sources. Gzip is informational; it was not the optimization objective. Python and its standard JSON implementation remain the common execution environment used by all compared programs.

1.3 New and inherited components

The initialized scalar iterator, Choice-DI logic, arithmetic bootstrap, uniform predecessor packets, unary/triplicate formula representation, arity-based scope traversal, and generic Python representation profiles are inherited from [2, 3, 4, 5, 6]. The new mathematical step restricts direct initializers and proves the restriction admissible with an independently constructed zero. The new certificate protocol uses that data field in place of an arithmetic tag. The last source reduction shares the already required assumption assignment across its three cases; it introduces no new rule.

An explicit ablation isolates the contribution of initializer dispatch. Restoring only the old `m is None` discriminator in the final three programs gives respectively 574, 566 and 606 bytes. The new discriminator saves exactly eight bytes, two tokens and three AST nodes in each corresponding program. These newly constructed ablations are separate from the seven driver-only controls and are included in `reference/ablations/`. Thus the larger total gain against the supplied corpus is not attributed entirely to the normalization theorem.

A repository-relative audit found no initializer-based computation dispatch among 370 distinct Python sources and 386 distinct textual sources. Earlier drafts do use proved nonempty representations of zero for witnesses and predecessors. That general technique is credited; its application to initializer dispatch and the simultaneous repair here are the new step. No priority claim outside this corpus is made.

2 The complete native theory

Here iPRA means full first-order intuitionistic arithmetic in a language of fixed finite primitive-recursive descriptions, with their defining equations and quantifier-free induction. Atomic induction is a customary equivalent formulation in the full primitive-recursive language; see Salehi's definition [9]. The internal conversion needed for our smaller language is proved in the attributed appendices.

2.1 Numerical expressions and restricted computation

There is one numerical sort, with finite terms

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only v in h is bound. All other arguments of h are passive parameters, fixed during iteration. Each fixed previously constructed update description gives a first-order function symbol; there is no quantification over functions and no uniformly total evaluator of all program codes.

There are exactly two nonlogical computation patterns:

$$I(0, a; v.h) = a \qquad \text{if } a \text{ is not the literal zero term,} \tag{R0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v] \qquad \text{if neither } n \text{ nor } a \text{ is literal zero.} \tag{RS}$$

The side conditions are metasyntactic. A permitted term may denote zero. Substitution in the update is capture avoiding. The restriction makes arbitrary numerical substitution an admissible transformation rather than a closure property of the displayed primitive axiom list: a substitution producing a forbidden literal zero is repaired by the next section's construction.

2.2 Object formulas and auxiliary closures

Object formulas are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A.$$

An auxiliary closure $\mathsf{Q}_y B$ may occur in contexts and certificates. It may also be admitted as a definable formula constructor, since it is interderivable with $\mathsf{D}_{u,y} B$ for unused u. Ordinary readings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

Contexts are finite reusable lists. The designated native contradiction is $0 = S0$. No standard propositional connective or quantifier is a separate native primitive.

2.3 All primitive logical rules

Hypothesis, equality transport, and closure specialization are

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{Hyp, Tr, QE}$$

F is a fixed finite native formula template; the transport rule permits transport through binders. Paired elimination is

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenparameter b is absent from Γ, A and the fixed parameters of the matrix. Paired introduction has the two alternatives

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}, \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}. \tag{DI-case, DI-progress}$$

Progress is restricted to a literal equation E, not a quantified formula or an equation beneath a binder. In both alternatives z is fresh for the retained context, witness and passive parameters, and the displayed local assumption is discharged.

The computation equations can be axiom leaves. The implementation uses the equivalent local-assumption form

$$\frac{\Gamma, E \vdash A}{\Gamma \vdash A}\quad (E \text{ a permitted R0 or RS instance}). \tag{Supply}$$

A computation leaf is Supply followed by Hyp; Supply is an axiom leaf followed by admissible cut. These are alternative proof layouts for the same theory, not extra arithmetic axioms.

Weakening, exchange, contraction, renaming, capture-free numerical substitution, and ordinary-assumption cut are admissible. They follow by translating to the unrestricted identical-sequent calculus, applying the usual finite structural transformation, and repairing computations. There is no primitive general induction, excluded middle, equality reflection rule, successor injectivity rule, reflexivity axiom, or coverage axiom. Their needed intuitionistic or arithmetic instances are derived as described below.

3 Using the initializer to select computation

The initialized scalar iterator has the usual equations

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{I0, IS}$$

The following restriction removes a computation opcode from the certificate interface without changing these operations or the native consequence relation. It concerns the *syntax* of an initializer, not whether that initializer denotes a positive natural number.

A fixed nonempty representation of zero. Let

$$C := I(S(S0), S0; v.0). \tag{Auxiliary zero}$$

One successor computation, at predecessor $S0$, proves

$$C = 0. \tag{Zero certificate}$$

The predecessor and initializer in this computation are both nonempty canonical terms. The update is the constant zero, so the right side of the computation is exactly zero. This construction does not use a previous computation with initializer zero, a zero-predecessor repair, an induction, or a presumed identity for a new numerical constructor.

Restricted computation system. In canonical numerical serialization, use scalar 0 for zero and nonempty arrays for successor and iteration. Consider the same native calculus with its direct computation leaves restricted as follows:

- I0 may be used directly only when its initializer is not literal zero.
- IS may be used directly only when both its initializer and its supplied predecessor are not literal zero.

The predecessor restriction is already present in the inherited predecessor-specified interface. The new restriction is on the initializer. A parameter or a nonempty term is permitted even when its value happens to be zero. In particular, C is permitted. Keep the ordinary native equality-transport rule and all other logical rules unchanged.

Admissibility theorem. Every unrestricted I0 and IS instance has a finite derivation in this restricted system. Consequently the restricted and unrestricted calculi prove exactly the same native sequents, with exactly the same numerical and formula languages.

Proof. Instances satisfying the stated syntactic conditions are direct. For a base computation with initializer zero, take the fresh-variable equation motive

$$E(w) := \big(I(0, w; v.h) = w\big).$$

The restricted base equation proves $E(C)$. Equality transport along the independently proved $C = 0$ gives $E(0)$, which is precisely the required unrestricted base instance.

For a successor computation with permitted predecessor n but zero initializer, take

$$E(w) := \left(I(Sn, w; v.h) = h[I(n, w; v.h)/v]\right).$$

The restricted successor equation proves $E(C)$, and the same transport gives $E(0)$. The displayed variable w is fresh for n, h, and all retained assumptions. Only the indicated initializer occurrences are substituted; unrelated zero occurrences or passive arguments in h remain fixed.

For a zero predecessor but a permitted initializer a, use instead

$$E(w) := \left(I(Sw, a; v.h) = h[I(w, a; v.h)/v]\right).$$

Again $E(C)$ is a permitted successor computation and transport gives the desired instance $E(0)$.

If both predecessor and initializer are zero, a single shared motive repairs them simultaneously:

$$E(w) := \left(I(Sw, w; v.h) = h[I(w, w; v.h)/v]\right).$$

Both supplied values in $E(C)$ are nonempty, and transport along $C = 0$ gives exactly $E(0)$. Bound state variables are renamed before these substitutions, so no capture occurs.

Each repaired computation leaf therefore uses one permitted computation for $C = 0$, one permitted computation for $E(C)$, and one transport inference. Replacing computation leaves in a native derivation proves the inclusion from the unrestricted calculus to the restricted one; the reverse inclusion is immediate. Contexts and final sequents are unchanged throughout. □

This is a three-node repair when computation leaves and transport are counted as one node each and the proof of $C = 0$ is copied. If a tree derivation has N nodes and r excluded computation leaves, the stated replacement has exactly $N + 2r$ nodes. If a DAG proof retains one shared node proving the closed lemma $C = 0$, the repaired part instead has $2r + 1$ nodes, giving $N + r + 1$ distinct nodes when $r > 0$. A certificate format with persistent proved assumptions can implement corresponding sharing by supplying this closed computation once before the affected continuation. These are counts for the displayed construction, not lower bounds on shortest certificates or claims about a particular producer's serialization.

Why the opcode can disappear. Write a four-field packet as $[m, s, u, t]$. The established elimination branch uses a truthy major m. In a computation packet, let s be the predecessor flag/value, u the initializer, and t the update. A truthy u selects the computation branch. The checker constructs the source counter as s when it is the zero flag and as Ss otherwise, and generates I0 or IS. A paired-introduction packet puts zero in the unused u field. Thus the dispatch can be written

```
if m:
    ... elimination ...
elif u:
    ... computation ...
else:
    ... paired introduction ...
```

The direct computation branch can no longer receive a literal-zero initializer. The admissibility theorem supplies exactly that missing case. No test that u denotes a nonzero number is performed or needed.

The fresh-name obligation on noncanonical packets. The dispatch change admits computation packets with any false-valued m, including the atomic-progress marker $m = 0$. In the common continuation, that marker can cause the generated hypothesis to be opened at the new rigid parameter b, rather than at literal zero. It would be incorrect to assert that the resulting $H(b)$ is uniformly true. Use the existing zero-instance invariant instead:

$$\vdash H(\vec{0}), \qquad \vec{b} \cap \mathrm{FV}(\Gamma, A) = \varnothing.$$

Here $\vec{b}$ includes the designated new integer name b and every additional negative fractional name in the newly admitted unit interval that occurs in the finite raw certificate. Such names must not be silently identified with the designated integer. Given a finite derivation of $\Gamma, H(\vec{b}) \vdash A$, rename its later eigenparameters if necessary, substitute zero for every member of $\vec{b}$, and cut the resulting assumption $H(\vec{0})$ against its computation derivation. This gives $\Gamma \vdash A$. The old-floor validation of the retained goal and context is what establishes the displayed freshness condition. Consequently the new raw packet possibilities are sound under the same zero-completion and opening lemmas as the predecessor-specified kernel. They do not justify silently strengthening a known zero instance into a uniform computation theorem.

Interpretations and the scope of the improvement. The theorem preserves every native sequent through the identity map on object expressions. It therefore transfers the inherited whole-domain bi-interpretation and equiconsistency with iPRA without a new domain, new equality relation, or new primitive numerical operation. The contribution is a removable syntactic restriction and a data-driven certificate dispatch. It is an implementation simplification, not a new arithmetic foundation or a proof of absolute minimality.

Relative to the inspected pinned draft collection, the new step is using the initializer itself to discriminate computation packets and proving that the resulting missing computation instances are admissible. The broader technique of replacing an unavailable zero representation by a proved nonempty representative is already used in the supplied drafts, including their predecessor and witness repairs. The present argument adapts that attributed technique to eliminate the computation opcode. No priority claim outside the inspected corpus is made.

4 Full iPRA and both interpretation composites

The new normalization preserves every native sequent by the identity map. It remains necessary to specify the inherited bridge: agreement on standard numerals or equiconsistency alone would not suffice.

4.1 Recovering the ordinary logical operations

Once the omitted computations have been restored, put

$$C(v, a, b) = I(v, a; u.b), \quad Z(v) = C(v, 1, 0), \quad p(v) = I(v, 0; u.Z(u)), \quad 1 = S0.$$

The use of C here for the conventional three-argument selector is separate from the closed auxiliary-zero constant of the preceding section; its arguments distinguish the two notations. Computation gives $C(0, a, b) = a$, $C(Sv, a, b) = b$, $p(1) = 1$, and $p(2) = 0$. For equations define

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad M_v(s = t, r = q) = (C(v, s, r) = C(v, t, q)).$$

Extend through paired binders by finite syntax recursion, aligning bound variables and using the proved padding equivalence $A \dashv\vdash \mathsf{D}_{u,y} G_y A$ when binder depths differ. The full recursive definition and its proof are in Appendix B. It gives

$$\vdash G_0 A, \quad G_{Sv} A \dashv\vdash A, \quad M_0(A,B) \dashv\vdash A, \quad M_{Sv}(A,B) \dashv\vdash B.$$

With all new binders fresh and u unused, define

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A,B), & A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x), & A \vee_* B &= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), & \bot_* &= (0 = 1). \end{aligned}$$

All ordinary intuitionistic introduction and elimination rules follow from DI-case, DE, QE and these endpoint identities. The zero conjunct in the universal definition is necessary. Conjunction projections use indices one and two. Native explosion first gives every equation by transport through the selector and then every formula by syntax induction. These constructions preserve arbitrary retained contexts.

DI-progress with an unused witness, followed by DE and QE, gives contextual equation induction. DI-case similarly gives zero/successor cases for arbitrary native formulas. The resulting logic is full intuitionistic first-order logic; no decidability principle for arbitrary quantified formulas is introduced.

4.2 Arithmetic and whole-domain interpretation

The included arithmetic development has an explicit dependency order: scalar predecessor; truncated subtraction and a characteristic of actual equality; quantifier-free induction by characteristic equations; then pairing and the compilation of arbitrary fixed primitive recursions. No pairing or equality detector is assumed in the predecessor bootstrap. All inductions keep passive parameters fixed. The equalities are proved internally for arbitrary domain elements.

If the target description is $f(0,\vec{a}) = g(\vec{a})$ and $f(Sn,\vec{a}) = h(n, f(n,\vec{a}),\vec{a})$, the proved pairing π and projections L, H give

$$\begin{aligned} V(n,\vec{a}) &= I\big(n, \pi(0, g^*(\vec{a})); u.\pi(SL(u), h^*(L(u), H(u), \vec{a}))\big), \\ f^*(n,\vec{a}) &= H(V(n,\vec{a})). \end{aligned}$$

Equation induction proves $L(V(n,\vec{a})) = n$ and the required defining equations. This is recursion on a fixed finite description in the metatheory, not a total numerical evaluator of every code.

Theorem 4.1 (Bi-interpretability and equiconsistency)**.** *The restricted native calculus presents a theory bi-interpretable and equiconsistent with full first-order iPRA. Both numerical interpretation domains are the entire domain, equality is actual equality, and both domain isomorphisms for the composites have graph $x = y$.*

Proof. First recover the unrestricted native consequence relation by Section 3. Interpret each scalar iterator in iPRA by primitive recursion with an update that ignores its counter. In the opposite direction use the displayed compiler and the internally proved arithmetic facts. Ordinary logical rules are recovered by the macros.

For each numerical composite, the translated operation satisfies the original defining equations. Internal equation induction proves uniqueness of that fixed recurrence with its parameters held fixed. Induction on the finite operation description proves equality of each operation with its double translation, including coherence under substitution of actual arguments.

The formula composites include every native formula, not only formulas in the image of the ordinary logical translation. In particular,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big),$$

and the analogous closure composite holds. Forward, DE supplies the fresh witness and its clause; under the zero instance use QE, and then introduce universal, implication and existential. Backward, eliminate the existential, retain the zero instance in DI-case, and obtain each successor instance by implication and universal elimination. External induction on formulas finishes both composites.

In the ordinary realization, include the explicit falsity bridge $(0 = S0) \to \bot$ as well as numerical coverage; the native derived explosion and cases supply them. Positive equations alone would not justify this bridge. Contradiction is preserved in both directions, so the effective finite proof translations establish equiconsistency. This is not a claim that either theory proves its own consistency. □

5 The complete checker and its interpretation

The recommended program below is the entire trusted Python source. No producer, evaluator, theorem database, measurement script or service is imported. It uses the inherited arity representation [2] with the new initializer dispatch and shared assumption assignment.

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <=
    a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t,
    2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == m) and k)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

5.1 Input, result and execution boundary

Input is one JSON array. Its final element is the requested closed formula; earlier four-field packets are consumed backwards. An unused prefix is inert. A successful process exits zero and prints exactly `ACCEPT`. An input or checking exception rejects with nonzero exit status; the traceback is diagnostic. Merely finding the word `ACCEPT` in arbitrary output is not the verdict convention. There are no trusted Python `assert` statements, and the same inference checks run under `-O`.

5.2 Sorted array interpretation

Canonical native formulas use

$$\mathsf{D}_{x,y}B \longmapsto [B], \qquad s = t \longmapsto [s,t], \qquad \mathsf{Q}_y B \longmapsto [B,B,B].$$

Only the last field of a general ternary formula is its matrix; the first two are annotations. Numerical terms use scalar zero, unary successor and ternary scalar iteration. Both interpretation and canonical encoding are sorted: the same array may have different meanings at a numerical and a formula position.

Arity	Numerical meaning	Formula meaning
0	0	$0 = 0$
1	S of its child	D of its matrix
2	0; annotations ignored	Equality of the two terms
3	$I(n, a; v.h)$	Q of the last field
≥ 4	0; children ignored	$0 = 0$; children ignored

The scalar numerical coordinates $0, 1, 2$ all denote zero at the numerical root depth three. Under p protected numerical slots, coordinates $3 + 2j, 4 + 2j$ name slot j, counted from the innermost slot, for $0 \leq j < p$. Negative admitted coordinates name rigid numerical parameters. Formula root depth is $2p$, and numerical root depth is $3 + 2p$. Successfully admitted scalar padding at formula positions denotes the fixed truth $0 = 0$.

In a unary numerical successor, its child is decoded under two dummy zero slots. In a ternary iterator, counter and initializer are decoded under one dummy zero slot, while the update is decoded under its genuine state slot. In a paired formula the two slots are y, x; in a closure the one slot is y. Thus the single child-depth update

$$d \longmapsto d + 5 - \text{arity}$$

gives the required increments $4, 3, 2$ at arities $1, 2, 3$. Canonical encoding shifts variables past the indicated dummy slots. Rigid negative names do not shift. The named producer implements these conventions independently of the kernel.

At larger arities the entire expression already denotes a constant. Traversing an ignored annotation can reject it; a successful traversal does not change that constant interpretation. In particular, zero or negative depth increments inside such annotations do not authorize a claim that all raw trees are idempotently revalidated. Only the semantic opening statement at meaningful sorted roots is needed.

5.3 Opening and freshness

At global floor k, the opener preserves a scalar in the interval $k \leq a < d$. At a numerical position an escaping scalar is the distinguished substitution hole, and its image is checked at numerical root depth three before insertion. A missing image at a used numerical hole rejects. At a formula position scalar padding instead erases to truth. Induction on a successfully traversed finite tree, using the preceding table, proves capture-free semantic opening at every meaningful position.

An inserted numerical image has no unbound positive coordinate at its own root. Its free parameters are rigid negative names. Hence copying it beneath extra protected slots does not capture its free parameters; its internal binders retain their indices. Raw equality of successfully checked expressions consequently implies equality of decoded meanings. This statement does not identify different numerical programs merely because they happen to have the same standard value.

Each non-hypothesis continuation decreases k. The integer new floor is the designated fresh parameter. Previously checked relevant goals and retained hypotheses mention no newly admitted name: their earlier no-image checks would have rejected such an occurrence. If noninteger JSON numbers occur, any additional names in the newly admitted interval are separate fresh parameters, not aliases of its integer endpoint. Only finitely many occur in a finite certificate. The known-zero-instance argument substitutes zero for all of them when necessary.

5.4 The three packet cases

Write a packet as $[m, s, u, t]$. A truthy major m takes priority. The checker proves it in the old context before proceeding. If it is the indicated equality $[s, u]$, and the current goal is the template opened at u, the goal changes to its opening at s. For a canonical closure $[t, t, t]$, the source field must denote the zero instance; the same operation implements QE. A mismatch leaves the goal unchanged. Either way a proved major is required, so a failed major does not become a harmless no-op.

The exported local expression is $m * 3$. A unary paired major exports its three-copy closure, with the witness opened at the fresh name. Every other nonempty array major exports a high-arity truth. Relevant scalar majors cannot create arbitrary unproved consequences: their only admitted formula interpretation is truth. The joint profile's additional iterable aliases are covered by its inherited decoder.

For a false-valued m and truthy u, the initializer selects computation. The packet constructs

$$\begin{cases} [\,[s, u, t], u\,], & s \text{ false-valued}, \\ [\,[[s], u, t],\ i(t, [s, u, t], 3)\,], & s \text{ truthy}. \end{cases}$$

The first is a zero computation; the second constructs an actual successor counter. When an update uses its state, filling it checks the entire recursive source, including its update, at numerical root depth three. This prevents an escaping update coordinate from being mistaken for an extra state variable. If the update does not use its state, the final generated equation still validates its meaningful contents. The inherited zero-instance proof applies after setting newly admitted names to zero.

When both m and u are false-valued, the packet is paired introduction. The unary goal is opened at the supplied witness t, retaining the index slot. Its successor instance becomes the next goal. Marker $m = 0$ retains the arbitrary-index instance only for a literal equation; the case marker $m = []$ retains the zero instance. Compound formulas get the zero instance. The literal-equation restriction therefore remains active. Exact unary unpacking rejects a binary equation as a paired introduction; the undersupplied empty-goal case has the harmless truth interpretation.

The last assignment shares construction of the local hypothesis across these cases. Python evaluates only the selected branch of the conditional expression. In the introduction branch the named assignment updates the goal and hypothesis to the same opened matrix, just as the separate assignments did before hoisting. The equality-transport Boolean expression in the byte and preferred profiles likewise changes no canonical proof: object and closure matrices are nonempty arrays. If a noncanonical transported source is false-valued, the expression keeps the original goal, imposing the stronger unchanged obligation.

5.5 Why a generated hypothesis need not be universally true

For fresh names $\vec{b}$ absent from Γ, A, the following rule is admissible:

$$\vdash H(\vec{0}), \quad \Gamma, H(\vec{b}) \vdash A \quad \Longrightarrow \quad \Gamma \vdash A.$$

Substitute zero for the fresh names in the second finite derivation, renaming later eigenparameters first, and cut in the first proof. No consistency or truth oracle is used.

This matters for the new dispatch, since arithmetic can now be selected with the otherwise-progress marker $m = 0$. For example the packet data

$$m = 0,\ s = 0,\ u = 3,\ t = 0$$

can generate, at the fresh parameter b, the decoded assumption

$$H(b) : I(0, 0; v.0) = b.$$

Its zero instance is a computation theorem; its instance at one is false in the standard model. The retained goal is validated before b exists. The tests accept an independent valid continuation after this supply and reject an attempted closed false consequence. The soundness argument is the admissible fresh-name rule, not a mistaken claim that every generated open hypothesis is uniformly true.

5.6 Soundness, completeness and profiles

Theorem 5.1 (Checker correspondence)**.** *Every successful finite run of the explicit-list checker has a decoded conclusion derivable in the native iPRA presentation. Every finite canonical native derivation has a finite certificate after the displayed computation normalization, under the usual unbounded-resource mathematical interpretation of the checker.*

Proof. For soundness, induct on the finite successful checking trace. Hypothesis closure uses semantic opening. The major branch uses the inductively proved major, transport or QE when their raw match holds, and DE or truth weakening for the export. Introduction uses the exact unary shape, the fresh-name invariant and the correct case/progress assumption. Computation uses its derivable zero instance and the fresh discharge lemma. The final continuation opening realizes the displayed successor or witness substitution. Hoisting the hypothesis construction preserves the evaluated branch and its state changes.

For completeness, first replace each excluded computation leaf by the three-node zero-certificate repair. Use canonical arity encodings, fresh negative labels, and the inherited backward packet serialization of Hyp, Tr, QE, DE and the two introductions. Every remaining computation has a truthy initializer and, in its successor case, a truthy predecessor. It therefore reaches the selected branch and generates its exact computation equation. Hyp closes that continuation. All retained contexts and conclusions are preserved by this finite translation. □

The byte profile's predicate `a*0 == []` distinguishes lists among successfully tested JSON values. Strings and numbers produce different zero products; mappings and null raise errors. It is the inherited compact spelling of a list test, not proof-supplied execution. The preferred explicit list predicate makes this intent clearer.

The joint profile tries iterable traversal and falls back to scalar handling on an exception. It accepts some additional harmless iterable aliases. Nonempty strings may recurse indefinitely in an ideal unbounded-stack execution; actual CPython rejects through its recursion limit and subsequent scalar type failure. Its simultaneous record must therefore be read as a fixed-runtime comparison and canonical-proof completeness statement, not an ideal all-JSON termination theorem. The explicit-list and zero-product profiles avoid that malformed-string qualification. All three remain subject to ordinary physical resource limits on large genuine proofs, as do the supplied baseline programs.

6 Executable evidence and reproducibility

Verification item	Result
Named and adversarial certificate cases	36
Complete CLI runs, three profiles and ordinary/`-O`	216
Expected successful runs	108
Expected rejected runs	108
Verdict mismatches	0
Frozen Python source byte strings remeasured	370
Driver-only matched controls reconstructed	7

The positive certificates include the auxiliary zero itself, each repaired arithmetic boundary, an update using its state, equality transport through a closure, literal-zero existential witnesses, actual equation induction using its induction hypothesis, and nested contexts. The negative cases include compound induction, escaping names, false equations, malformed updates, incorrect endpoints and an unproved major. New tests distinguish the rejected old zero-initializer packet from a successful proof of the same theorem through the zero certificate.

The producer is a separate untrusted program. It constructs named proof trees, applies the simultaneous three-node normalization, and serializes them without importing a checker. It is a producer for the supplied examples and the displayed native proof-tree grammar, not a completed automatic compiler from arbitrary conventional iPRA input. General completeness is the written proof transformation.

Finite tests verify implementations and counterexamples; they do not prove universal soundness or bi-interpretability. The written arguments and the scoped independent review are included. There is no proof-assistant certification in this package.

Run the complete offline reproduction with

```
python3 reproduce.py
python3 build.py
```

The first command verifies the manifest, remeasures all frozen source bytes and controls, regenerates the named certificates, and replays the selected implementations in ordinary and optimized Python. The second compiles the standalone LaTeX source and requires a normal TeX Live installation. The archived proof-checker programs import only Python's standard JSON module. Historical corpus sources are parsed for measurement and are not executed by the corpus remeasurement.

7 Two reductions that were rejected

A smaller apparent branch count is not sufficient. We retain two separate obstructions because they identify actual mathematical and proof-completeness failures.

7.1 Unconditional coercions lose reflexivity

An experimental opener replaced every exposed numerical coordinate by a binary projection box $[t, 0]$. Numerically that box was intended as identity; at a formula position it was interpreted as the zero test $t = 0$. This seemed to remove a sort-dependent branch. But the computation rules supplied no projection equations.

For the precise archived experiment define a syntactic valuation V: every nonbinary formula has value true, while

$$V([L,R]) \quad \Longleftrightarrow \quad R = 0 \text{ literally or } L \text{ is a ternary array.}$$

Every successful opening preserves V: zero stays zero, an inserted box is binary rather than ternary, and existing array arities are preserved. Computation supplies have a ternary iterator on the left; major exports have arity divisible by three; equality transport and closure specialization preserve V; paired introduction has a unary conclusion. Induction on finite successful traces shows that every accepted closed goal has value true.

The experiment represented $S0$ by $[0,0,0,0]$. Its ordinary reflexivity equation $[S0, S0]$ has value false, so no finite certificate can establish it. Adding an outer projection box to both endpoints has the same problem. The experiment is not a delivered iPRA checker. This is a completeness obstruction, not a claim of an arithmetically false accepted theorem. It shows why a plausible semantic alias does not automatically provide a usable internal equality.

8 A finite-model obstruction to eager initialization

There is a tempting way to shorten the computation clause of an iteration checker. Instead of preserving an independent initializer at counter zero, apply the update immediately:

$$F(0,a;v.h) = h[a/v], \tag{1}$$
$$F(Sn,a;v.h) = h[F(n,a;v.h)/v]. \tag{2}$$

The same syntactic opening operation then appears in both computation clauses. This subsection proves that, with the stated remaining primitives, this change cannot present iPRA. The obstruction applies to relational interpretations and to finite-dimensional interpretations with quotients, as well as to translations by terms.

The precise proposed theory. Let the numerical terms be the finite expressions

$$t ::= x \mid 0 \mid S(t) \mid F(t,a;v.h).$$

In the last constructor, v is bound in the update h only. Every other free variable of h is a passive parameter. Nested occurrences of F are allowed, but this is a finite structural grammar: the update does not refer circularly to the operation currently being defined. Equivalently, one may use a staged first-order signature containing a separate operation symbol for each already constructed update description. Substitution is capture avoiding.

Let T_{e} use ordinary intuitionistic first-order logic with actual equality, the equations (1)–(2), the two boundary assertions

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad 0 \neq S0,$$

and induction on literal equations, with arbitrary retained assumptions and passive parameters. The countermodel below also satisfies the stronger assertion $\forall x\,(0 \neq Sx)$ and induction on *every* formula. It therefore covers any version obtained by strengthening the induction schema in that way. Successor injectivity is *not* an axiom of this proposed simplification.

Two-element interpretation. Take the ordered set $M = \{0,1\}$ with $0 < 1$, interpret zero by 0, and put

$$S^M(b) = 1 \quad (b \in M).$$

For a variable assignment ρ, interpret the eager constructor by

$$[\![F(n, a; v.h)]\!]_\rho := [\![h]\!]_{\rho[v \mapsto [\![a]\!]_\rho]}. \qquad \text{(Eager interpretation)}$$

The counter is deliberately ignored. This equation is a definition of the mathematical interpretation, not a new evaluator trusted by the proposed proof checker.

Monotonicity lemma. Every numerical term is monotone in all its free variables under this interpretation.

Proof. Use structural induction on the finite term. Variables are projections, zero is constant, and successor is the constant function 1. For the remaining case let

$$A(\rho) = [\![a]\!]_\rho, \qquad H(b, \rho) = [\![h]\!]_{\rho[v \mapsto b]}.$$

After renaming the bound variable if necessary, the induction hypothesis says that A is monotone and H is monotone jointly in b and its passive parameters. If $\rho \le \sigma$ coordinatewise, then

$$H(A(\rho), \rho) \le H(A(\sigma), \sigma).$$

This is exactly the assertion for $F(n, a; v.h)$. Dependence on the counter term may disappear, which causes no problem for monotonicity. Nested eager binders are handled by the same structural induction. □

For completeness, the usual substitution lemma holds for this interpretation: evaluating a capture-free term substitution is the same as modifying the corresponding free-variable assignment. Its proof is another structural induction. In the binder case first rename v away from the substituted variable and replacement term, and then apply the induction hypotheses to a and h. Consequently the displayed interpretation respects the formal substitution in both computation schemas, including arbitrary fixed passive parameters.

Idempotence lemma. Every monotone map $H : M \to M$ is idempotent.

Proof. The possible pairs $(H(0), H(1))$ are precisely

$$(0, 0), \qquad (0, 1), \qquad (1, 1).$$

They are respectively the constant-zero map, the identity, and the constant-one map. Each satisfies $H(H(b)) = H(b)$ for both $b \in M$. □

Finite-model theorem. The structure M is a model of T_{e}, and remains a model if literal-equation induction is replaced by full first-order induction.

Proof. Fix arbitrary values of all passive parameters and let H be the resulting unary interpretation of the update h. By the monotonicity lemma H is monotone, and therefore idempotent. If a^M is the value of the initializer, the zero clause reads

$$H(a^M) = H(a^M).$$

The left side of the successor clause is again $H(a^M)$, because the counter is ignored; its right side is $H(H(a^M))$. These agree by idempotence. The substitution lemma justifies these calculations for the schemas as written, not merely for closed numerical instances.

Coverage holds because the elements are 0 and $S0 = 1$. Also $Sx = 1 \neq 0$ for each x. To check induction, fix any formula $A(x, \vec{p})$ and an assignment of its passive parameters. If $A(0, \vec{p})$ and

$$\forall x\,(A(x, \vec{p}) \to A(Sx, \vec{p}))$$

hold, the step at $x = 0$ gives $A(1, \vec{p})$. These are all elements of the domain, so $\forall x\, A(x, \vec{p})$ holds. This argument works for arbitrary formulas and under any fixed retained context. The classical structure is also a one-world intuitionistic Kripke model, so all intuitionistic first-order rules with equality are valid in it. □

The native paired-binder presentation is covered. If the eager arithmetic is instead combined with the inherited Eq/Q/D presentation, interpret its clauses by

$$Q_y B(y) := B(0) \to \forall y\, B(y), \qquad D_{x,y} B(x, y) := \exists x\big(B(x, 0) \to \forall y\, B(x, y)\big).$$

Hypothesis use, equality transport, Q specialization from its zero instance, and fresh-witness D elimination are sound in this ordinary reading. For case introduction, choose the supplied witness t. The retained base assumption supplies the $y = 0$ instance, while the fresh-successor premise supplies the $y = 1 = S0$ instance. For progress introduction, the retained base assumption and the forward step imply every instance by the induction just verified. The usual freshness conditions keep the witness and passive parameters fixed in these arguments. Thus neither removing ordinary connective primitives nor packaging induction as the native progress rule removes the countermodel.

Corollary: no interpretation of iPRA. There is no ordinary finite-dimensional first-order interpretation of iPRA in T_{e}, even if its interpreted domain is a definable subset of M^d modulo a definable equivalence relation. Consequently T_{e} cannot satisfy the required bi-interpretability condition.

Proof. An interpretation would turn every model of T_{e} into a model of iPRA. In the displayed two-element model, a definable subset of M^d has at most 2^d elements, and a quotient cannot increase that number. A finite collection of interpreted sorts does not change finiteness. But iPRA proves that all standard successor numerals are distinct, using successor injectivity and zero–successor separation; hence it has no finite model. This is a contradiction. The same argument permits a fixed finite tuple of parameters in the interpretation. □

Scope and relation to the earlier obstructions. This result concerns eager initialization $h(a)$ with an independent initializer a. It is not the existing almost-unary growth argument for a tied seed or a zero-seeded restricted iterator. The supplied `local_scopes` draft also already excludes relational bi-interpretability for an existential-free tied-seed theory, by finite witness extraction and finite-profile image counting. The present argument obstructs all ordinary finite-dimensional relational interpretations, including quotients, because it exhibits a finite model of this different eager theory. In particular it excludes even a one-way interpretation of iPRA; it is not a claim to be the first obstruction to relational bi-interpretability in this research program. It does not apply to a recursion update with a separately available counter argument, to the initialized iterator $I(0, a; h) = a$, or to the successor-seeded iterator $R(0, a; h) = Sa$. Those alternatives require their own analyses. Adding successor injectivity would exclude this countermodel, but would be an additional arithmetic axiom

and would not by itself prove a new primitive-recursion compiler or bi-interpretation. This obstruction rules out the particular apparent simplification; it is not a lower bound on all iPRA axiomatizations and is not a positive size improvement.

9 Assessment and remaining limits

The positive result is an exact admissibility theorem with an executable consequence: a computation tag disappears, the complete source becomes smaller, and all existing interpretation obligations are preserved by the identity translation. The independent zero certificate is the reason the syntactic restriction is legitimate. The shared assignment and inherited representation profiles contribute further implementation savings, which are separated from that theorem in the comparison.

The result does not establish an absolute simplest theory. It keeps the inherited arithmetic language, iterator, and logical reconstruction. Calling this a significant new arithmetic foundation would overstate the finding. The strongest simultaneous source comparison also uses the explicitly qualified generic traversal. A completely explicit-list implementation below every matched token/AST/byte minimum remains a stronger implementation target, even though its present byte size is already substantially smaller.

The next formalization target is the decoder/opening invariant together with known-zero-instance supply. The new normalization theorem is short and independent of the difficult scalar predecessor bootstrap, making it a convenient separately checkable contribution.

A Attribution for the complete inherited bridge

The following five appendices are reproduced from the arity-calculus archive in the pinned snapshot, where they retain their earlier provenance from generated-assumptions and scalar developments. Their text is unchanged. Statements such as "new" within an inherited appendix refer to its original report. They are supplied to make the logical and arithmetic bridge available in this package; they are not claimed as new results here. The exact input hashes and MIT No Attribution license are included in `reference/`.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse

direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [10] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{3}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{4}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{5}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{6}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{7}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (7) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{8}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (7); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{9}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (8), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{10}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{11}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (11). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (12) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (13) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (14)$$

The base is (13). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (15)$$

The first uses the comparator recurrence. The second uses (14) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (16)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (17)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (16) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [11]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{18}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{19}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{20}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{21}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{22}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (22) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{23}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{24}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{25}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (18). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (24); symmetry and (21) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{26}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{27}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{28}$$

To prove the range assertion, split (20). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (19) also give, for all u, v,

$$C(u,v,v) = v. \tag{29}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{30}$$

If $u = 1 = S0$, (25) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{31}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{32}$$

Using these equations and (22), we now derive

$$0 + y = y, \tag{33}$$
$$(Sx) + y = S(x + y), \tag{34}$$
$$d(Sx, Sy) = d(x, y), \tag{35}$$
$$d(Sa, a) = 1. \tag{36}$$

Each induction below has a single equation as its induction formula.

For (33), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (34), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (34) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{37}$$

Indeed, both sides are $S(y + r)$.

For (35), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (36). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (35) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)\big) = x. \tag{38}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (33) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (29).

For the step, assume (38) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (38) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (37) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (38). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{39}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (38). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (36) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{40}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{41}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{42}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (27), which is equivalent to $d(Sa,b) = 1$ by (30), and then to $a = b$ by (39). The range assertion is (26) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (41) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,\big(a=b\vee(a=b\to\bot)\big) \tag{43}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{44}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{45}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{46}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{47}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A=0\vee b_A=1\big)\quad\text{and}\quad\big(b_A=0\longleftrightarrow A\big). \tag{48}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (41)–(42). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (48) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (48), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [12]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, current draft collection*, commit `fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`. https://github.com/VladimirReshetnikov/ProveIt/tree/fb2dbd2a33c6b132671a54b029d0bf3cd98c633d/docs/drafts.

[2] *An Arity Calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip` in [1]. Source of the unary/triplicate representation, arity opening, and the five attributed mathematical appendices.

[3] *Uniform Predecessor Certificates for Intuitionistic PRA*, `ipra_uniform_predecessor_2026-11-10.zip` in [1]. Source of predecessor packets, supplied frontier implementations, and the named producer adapted here.

[4] *Constructed Computation*, `ipra_constructed_computation_2026-11-10.zip` in [1].

[5] *Numerical Axioms as Local Assumptions*, inherited in the reference material of [2, 4].

[6] *A One-State Recursor for iPRA*, `ipra_transient_seed_2026-11-10.zip` in [1].

[7] *Affine Depths for a Small iPRA Checker*, `ipra_affine_arity_2026-11-10.zip` in [1].

[8] *Local Scopes*, `ipra_local_scopes_2026-11-10.zip` in [1]. Includes the earlier relational obstruction for a tied-seed basis.

[9] Saeed Salehi. *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*. Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.

[10] Inherited scalar predecessor development, retained with attribution in `reference/scalar_predecessor.tex` of [2].

[11] Inherited forward-left equality-reflection development, retained in `reference/equality_reflection.tex` of [2].

[12] Inherited progress-closure pairing and primitive-recursion compiler, retained in `reference/primitive_recursion.tex` of [2].